

Architecture & Technology Stack

▼ Relevant source files

Backend/STUDER-MEDIA/Dockerfile

Backend/STUDER-MEDIA/main.py

Backend/STUDER-MEDIA/placeholder.txt

Backend/STUDER/Dockerfile

Backend/STUDER/pom.xml

Backend/STUDER/src/main/java/facu/studer/StuderApplication.java

Backend/STUDER/src/main/java/facu/studer/config/ApplicationStart.java

Backend/STUDER/src/main/java/facu/studer/config/I18nConfig.java

Docker/docker-compose.yml

Docker/nginx.conf

This page describes the multi-tier architecture of the STUDER platform, detailing the technologies used, the network topology, and the communication patterns between services. The system is designed as a distributed set of containerized services orchestrated via Docker Compose.

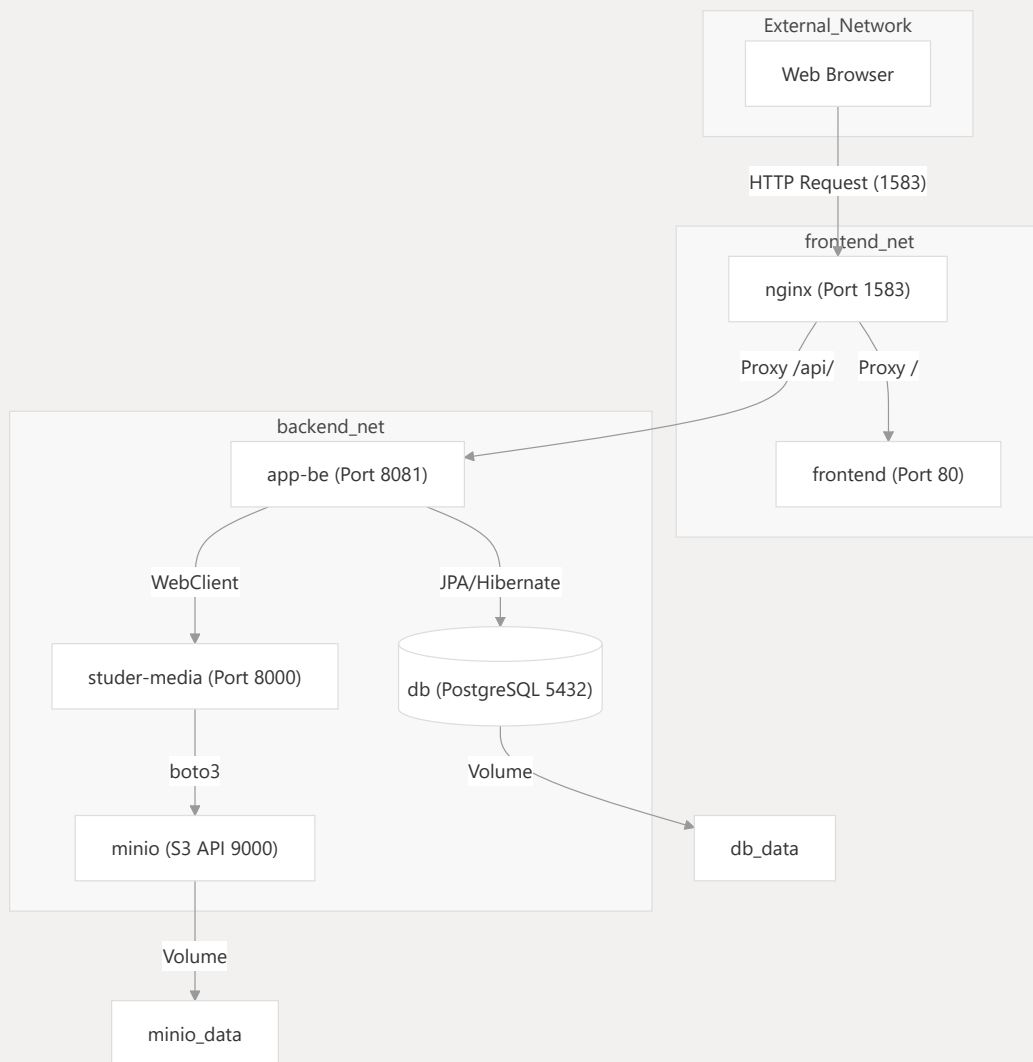
System Overview

STUDER utilizes a modern micro-monolith approach where a central Spring Boot backend coordinates with specialized services for media processing and object storage. The frontend is a decoupled Single Page Application (SPA) that interacts with the backend through a unified reverse proxy.

Component Diagram: Request Flow & Port Mapping

The following diagram illustrates how external requests are routed through the `studer_nginx` reverse proxy to the respective internal services.

Title: System Component Overview



Sources:

- `Docker/docker-compose.yml` 1-94
- `Docker/nginx.conf` 28-51

Technology Stack

Frontend: Angular SPA

The user interface is built using **Angular 19**. It is served via Nginx in production and communicates with the backend exclusively through RESTful APIs.

- **Port:** Internal **80**, mapped to **4200** in development `Docker/docker-compose.yml` 54-55

- **Network:** `frontend_net` `Docker/docker-compose.yml` 58

Backend: Spring Boot

The core business logic is implemented in **Java 17** using **Spring Boot 4.0.6**

`Backend/STUDER/pom.xml` 8-30

- **Key Dependencies:**
 - `spring-boot-starter-data-jpa` : For ORM and database interactions
`Backend/STUDER/pom.xml` 39
 - `spring-boot-starter-security` : For JWT-based authentication
`Backend/STUDER/pom.xml` 68
 - `springdoc-openapi` : For Swagger UI documentation `Backend/STUDER/pom.xml` 122
- **Port:** Internal `8081` , mapped to `8081` externally `Docker/docker-compose.yml` 32
- **Configuration:** Uses `prod` profile in containerized environments
`Docker/docker-compose.yml` 23

Media Service: FastAPI

A specialized Python microservice (`studer-media`) handles image processing tasks

`Backend/STUDER-MEDIA/main.py` 16

- **Framework:** FastAPI `Backend/STUDER-MEDIA/main.py` 16
- **Processing:** Uses **Pillow (PIL)** for generating image variants (original, avatar, webp, thumbnail) `Backend/STUDER-MEDIA/main.py` 7
- **Port:** `8000` `Docker/docker-compose.yml` 39

Infrastructure & Storage

- **Database:** PostgreSQL 16 for relational data storage `Docker/docker-compose.yml` 5
- **Object Storage:** MinIO (S3-compatible) for storing media files
`Docker/docker-compose.yml` 75
- **Reverse Proxy:** Nginx acts as the entry point, handling SSL termination (if configured) and path-based routing `Docker/nginx.conf` 33-50

Sources:

- Backend/STUDER/pom.xml 1-125
- Backend/STUDER-MEDIA/main.py 1-26
- Docker/docker-compose.yml 1-94

Network Topology & Communication

The system is partitioned into two Docker networks to isolate concerns and improve security.

Network Name	Purpose	Included Services
frontend_net	Public-facing communication	nginx , frontend
backend_net	Internal service communication	nginx , app-be , studer-media , db , minio

Service Interaction Flow

- Frontend to Backend:** The Angular app sends requests to `/api/`. Nginx routes these to `app-be:8081` Docker/nginx.conf 33-39
- Backend to Media Service:** When a user uploads a file, the `app-be` service communicates with `studer-media` via the internal URL `http://studer-media:8000` Docker/docker-compose.yml 30
- Media Service to MinIO:** The Python service uses `boto3` to upload processed images to `minio:9000` Backend/STUDER-MEDIA/main.py 36
- Data Persistence:** Both `db` and `minio` use Docker volumes (`db_data` and `minio_data`) to ensure data persists across container restarts Docker/docker-compose.yml 87-89

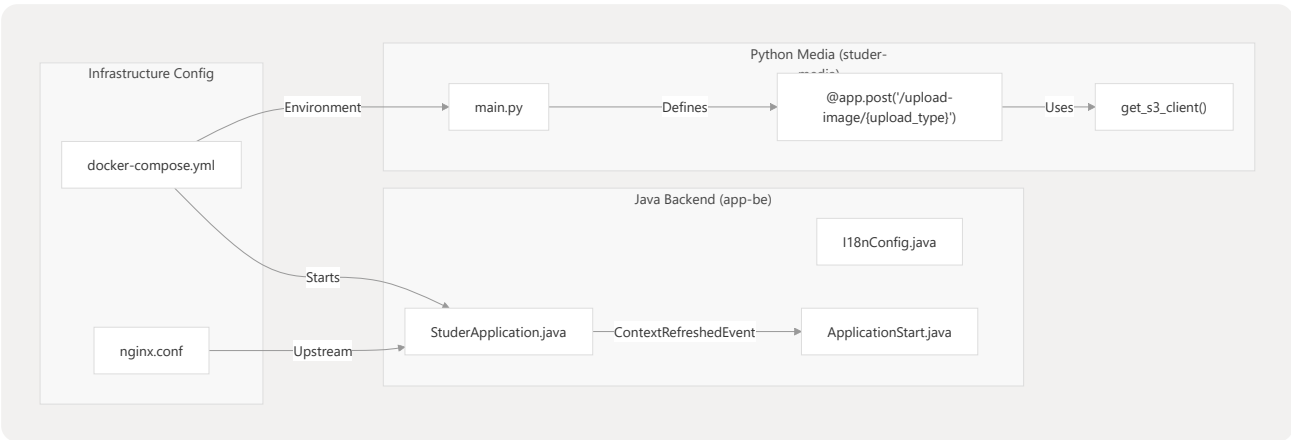
Sources:

- Docker/docker-compose.yml 90-94
- Docker/nginx.conf 18-26
- Backend/STUDER-MEDIA/main.py 18-22

Code-to-Architecture Mapping

The following diagram bridges the high-level architecture with specific code entities and configurations.

Title: Code Entity Architecture Mapping



Key Implementation Details

1. Backend Initialization

The Spring Boot application entry point is `StuderApplication.java` Backend/STUDER/src/main/java/facu/studer/StuderApplication.java 7 Upon startup, the `ApplicationStart` component listens for `ContextRefreshedEvent` to verify the application context is ready Backend/STUDER/src/main/java/facu/studer/config/ApplicationStart.java 17

2. Media Processing Pipeline

The `studer-media` service implements an automated bucket setup on startup via `create_bucket_if_not_exists()` Backend/STUDER-MEDIA/main.py 60 It configures a public read policy using `set_public_read_policy()` so that images can be served directly via URL Backend/STUDER-MEDIA/main.py 42

3. Localization Support

Internationalization is handled on the backend via `I18nConfig.java`, which defines an `AcceptHeaderLocaleResolver` to determine the user's language from HTTP headers Backend/STUDER/src/main/java/facu/studer/config/I18nConfig.java 15

Sources:

- Backend/STUDER/src/main/java/facu/studer/StuderApplication.java 1-17
- Backend/STUDER/src/main/java/facu/studer/config/ApplicationStart.java 52-152
- Backend/STUDER/src/main/java/facu/studer/config/I18nConfig.java 10-19
- Backend/STUDER-MEDIA/main.py 93-170